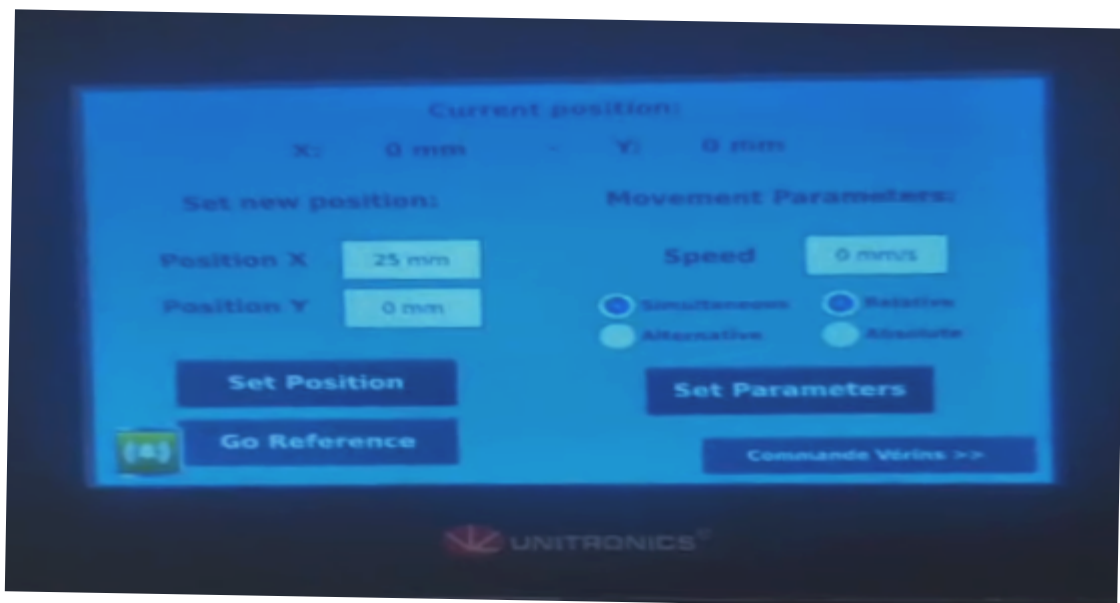


Modularisation du code MSP430 et correction de l'IHM

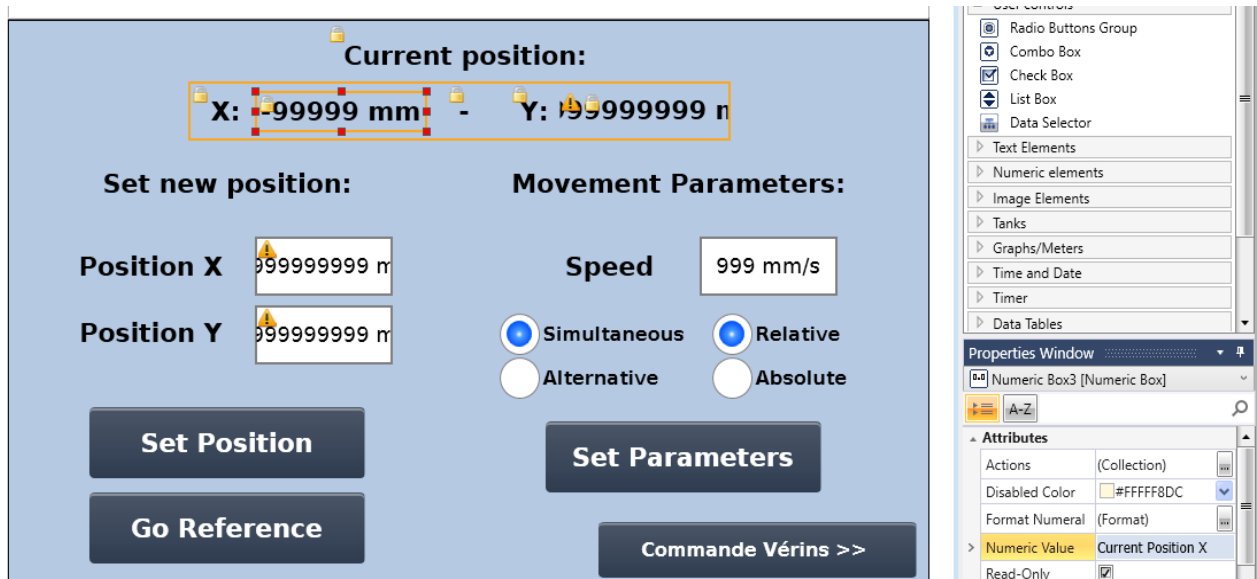
CHOU BRI Douae

1. Correction de l'IHM

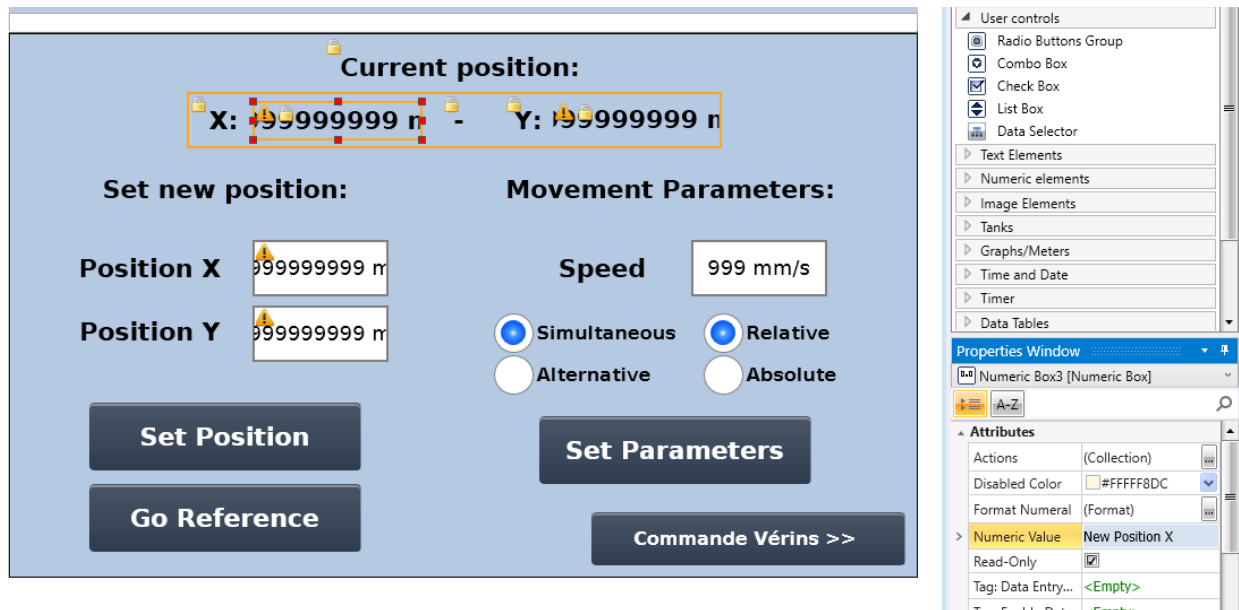
Auparavant, dans la section *Current position*, les valeurs X et Y ne se mettaient pas à jour, même après avoir modifié les positions via *Set Position*.



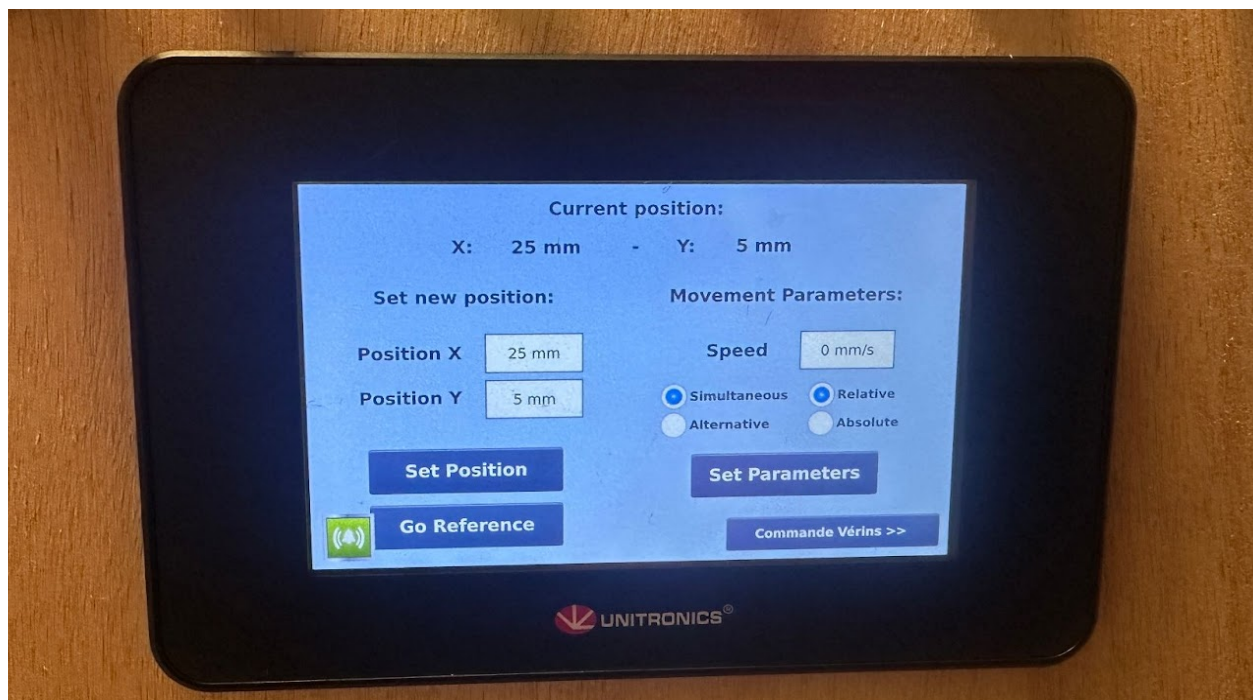
Le problème venait du fait que les variables utilisées (**Numeric Value**) pour afficher la position actuelle, **Current Position X** et **Current Position Y**, n'étaient définies nulle part dans le programme.



Pour corriger cela, les valeurs affichées dans la position actuelle ont été reliées aux variables **New Position X** et **New Position Y**, qui elles sont correctement définies dans les champs de saisie *Position X* et *Position Y* de la section *Set New Position*.



Grâce à cette correction, les valeurs X et Y se mettent désormais à jour correctement et varient simultanément dès qu'un changement est effectué dans l'IHM.



2. Modularisation du code MSP430

Avant modularisation, l'intégralité du programme était concentrée dans un seul fichier `main.c`. Ce fichier regroupait le contrôle moteurs, le décodage des commandes, la communication UART, les interruptions, les conversions mécaniques, la configuration GPIO, etc.

Le fichier atteignait **près de 498 pages**, ce qui rendait le code difficile à lire, maintenir ou déboguer.

Pour résoudre ce problème, une **réorganisation complète du projet** a été réalisée. Le programme est désormais séparé en plusieurs modules, chacun chargé d'une fonctionnalité précise.

Grâce à cette modularisation, le fichier `main.c` a été considérablement allégé : il ne fait plus que **67 pages**.

2.1 Nouvelle architecture du projet

La nouvelle structure du code repose sur des modules spécialisés :

→ `motor.c` / `motor.h` — Gestion matérielle des moteurs

Ce module contient :

- l'initialisation du timer TB0 (`motor_timer_init()`)
- le contrôle du timer (`timer_on()`, `timer_off()`)
- la configuration des GPIO moteurs (`motor_gpio_init()`)
- les fonctions d'activation/désactivation (`enableMotors`, `enableMotorsX`, `enableMotorsY`)
- le homing utilisant les capteurs de fin de course (`HomePositionXY`)
- les conversions mécaniques mm → pas (`mmToStep`, `stepToMm`)

Il regroupe ainsi tout le contrôle bas-niveau des moteurs pas-à-pas X et Y.

→ `command.c` / `command.h` — Interprétation des commandes

Ce module traite les commandes envoyées par l'IHM/PLC sous forme de chaînes UART :

- Commande de **position** : **P**xx-yy → Convertit mm → pas, choisit la direction, lance le mouvement, envoie <P1>.
- Commande de **vitesse** : **V**vitesse-simAlt-relAbs → Met à jour la vitesse, reconfigure le timer, envoie <S1>.
- Commande **Home** : **H** → Lance le retour automatique des axes à leur position d'origine.
- Sélecteur de commande : get_command() → Dirige vers la fonction adaptée (P / V / H)

Ce fichier regroupe tout le niveau logique du robot.

→ **uart.c / uart.h** — Communication UART

Ce module gère :

- la configuration complète des UART A0 et A1,
- les buffers de réception/émission (RX_Buffer, TX_Buffer),
- l'envoi non bloquant via interruptions (**uart_put_string()**),
- la réception caractère par caractère et la reconstruction automatique d'une commande.

Il prend en charge toute la communication série entre MSP430 et l'IHM/PLC.

→ **isr.c / isr.h** — Routines d'interruption

Ce module centralise toutes les ISR :

- **ISR Timer TB0 (moteurs)**
 - génère les pas moteurs (toggle STEP),
 - décrémente les pas restants,
 - arrête automatiquement le timer en fin de mouvement.
- **ISR UART A0/A1**
 - réception caractère par caractère,
 - détection du début (<) et fin (> ou CR/LF) de commande,
 - appel direct à **get_command()**.

- **ISR fin de course (Port 1)**

- arrête immédiatement l'axe concerné,
- remet la position à zéro (Home position).

Ces ISR permettent un fonctionnement asynchrone, fluide et totalement non bloquant.

2.2 Un fichier `main.c` plus simple et clair

Le fichier `main.c` ne contient plus que les actions essentielles :

- Initialisation des horloges
- Initialisation des GPIO moteurs
- Initialisation des capteurs de fin de course
- Initialisation UART
- Activation des moteurs
- Configuration des interruptions
- Boucle infinie vide (tout fonctionne par interruptions)

Exemple extrait du main :

```
motor_gpio_init();
motor_timer_init();
uart_init();
GPIO_enableInterrupt(FCX_PORT, FCX_PIN);
GPIO_enableInterrupt(FCY_PORT, FCY_PIN);
__enable_interrupt();
while(1){
    // Programme entièrement piloté par ISR
}
```

Le rôle du main est désormais de **configurer l'environnement**, puis de laisser les modules travailler en arrière-plan.

2.3 Résultats de la modularisation

Grâce à cette séparation :

- ★ `main.c` passe de 498 pages → 67 pages
- ★ Chaque fonctionnalité possède son propre module

- ★ Le code est lisible, structuré et évolutif
- ★ Les interruptions gèrent tout en temps réel
- ★ La communication UART est propre et non bloquante
- ★ Le mouvement moteur est sécurisé et centralisé
- ★ Cette modularisation constitue une amélioration majeure en termes de **clarité**, de **maintenance**, de **fiabilité**, et d'**extensibilité** du code.